用自然语言替代复杂代码

原创 天猫技术团队 大淘宝技术 2026年3月13日 01:34 浙江 470人



本文介绍了利用 AI平台，通过**自然语言描述业务规则 + 预置 AI Agent + 参数化调用**的方式，替代传统复杂代码（150+行）来解决电商促销中“多 SKU 凑单破价”资损风险巡检问题的实践方案；核心价值在于将高理解成本、难维护的硬编码逻辑，转化为可读性强、修改便捷、开箱即用的 AI 驱动分析流程，并在开发效率、维护成本、响应速度和业务价值上实现显著提升。

01 背景:一个棘手的业务问题

某业务场景中存在一个复杂的数据分析需求:多规格商品的组合计算。

场景举例:

```
某业务活动:针对商品进行优惠计算
商品有多个规格,价格差异大
- 高价规格:¥600+
- 低价规格:¥100+
- 活动优惠:每件固定金额
用户组合购买:
1个高价 + 多个低价
总优惠按件数叠加
高价规格按比例分摊优惠
场景:高价规格到手价可能低于预设阈值
```

业务需求:活动前自动检测,找出有风险的数据并预警。

02 痛点:传统方案太难维护

2.1 编写规则代码?150 行起步

传统方案需要编写精确的计算代码:

```
// 伪代码示意(实际更复杂)
function checkPriceRisk(item) {
  // 1. 筛选有效规格(库存>0,状态正常)
  const validSkus = item.skus.filter(s =>
    s.sellable_quantity > 0 && s.sku_status === '正常'
  );
  // 2. 找最高价和最低价规格
  const highSku = maxBy(validSkus, 'page_price');
  const lowSku = minBy(validSkus, 'page_price');
  // 3. 获取基准价格(可能需要调用服务)
  let basePrice = highSku.base_price;
  if (!basePrice) {
    basePrice = await queryBasePrice(item.item_id);
  }
  // 4. 循环计算不同组合数量
  for (let n = 1; n <= lowSku.sellable_quantity; n++) {
    const totalPrice = highSku.page_price + lowSku.page_price * n;
    const totalDiscount = item.discount * (1 + n);
    const highSkuDiscount = totalDiscount * (highSku.page_price / tota
    const finalPrice = highSku.page_price - highSkuDiscount;
    // 5. 判断是否超出阈值
    if (finalPrice < basePrice) {
      // 记录风险...
    }
  }
  // ... 还有很多边界处理
}
```

问题:

- 代码复杂:150–200 行,多层循环嵌套;
- 理解成本高:新人需要仔细读代码才能理解逻辑;
- 维护困难:规则变化(如改计算公式)需要改代码、测试、发布。

2.2 为什么是 AI?

观察上面的代码,其实逻辑本身不复杂,**核心是"多步骤的推理和计算":**

1. 筛选有效规格
2. 找最高价和最低价
3. 获取基准价格(可能需要调用工具)
4. 循环计算组合场景
5. 判断是否超出阈值

这种"理解任务 → 多步推理 → 生成结果"**的能力,恰好是 AI 大模型的优势。

理想方案:

```
用自然语言描述分析任务 → AI 执行推理和计算 → 返回结构化结果
```

但新问题来了:如何在业务系统中快速接入 AI 能力?

直接调用 LLM API 面临三大难题:

1. 参数复杂:model、temperature、max_tokens、system prompt、user prompt.....需要深入了解 AI 技术
2. Prompt 难写:要写出高质量的 Prompt 需要大量调试和经验积累
3. 维护困难:Prompt 写在代码里,每次调整都要改代码、测试、发布

AI 平台的解决方案:

- ✅ 预置 Agent:平台已经配置好专业的 Prompt 模板(如"数据分析师")
- ✅ 参数化调用:只需填充业务参数(job、data),无需关心底层细节
- ✅ 配置管理:Agent URL 在配置中心管理,版本切换无需改代码

简单说:AI 平台把复杂的 AI 能力封装成"填空题",我们只需填充业务参数即可。



解决方案:AI 平台让 AI 接入变简单

3.1 AI 平台优势?

AI 平台提供预配置好的 AI Agent供业务直接使用。

核心机制:

```
AI 平台
   ├─ 预置多种专业 Agent(数据分析师、文案助手、客服助手...)
   ├─ Agent 内置完整的 Prompt 模板
   ├─ Prompt 中预留参数坑位(${job}、${data}、${type}...)
   └─ 暴露标准化的 API 接口
业务系统调用
   ├─ 选择合适的 Agent
   ├─ 填充业务参数(job、data)
   └─ 获取分析结果
参数 → Agent → 结果
```

关键特点:

- **开箱即用:**Agent 已经优化好,无需从零写 Prompt
- **参数化:**只需填充业务参数,降低使用门槛
- **配置管理:**Agent URL 可配置,支持版本切换

- **持续优化:**平台持续优化 Agent,自动提升效果

简单理解:**AI 平台就像一个"AI 能力超市",我们选择合适的 Agent,填充参数就能用。**

3.2 如何使用?

AI 平台的核心优势是提供了预配置好的 Agent,我们只需要"填空",不需要从零写 Prompt。

Step 1:选择合适的 Agent

AI 平台上已经有丰富的预置 Agent,比如"数据分析"类 Agent:

```
角色定位:专业的数据分析师
核心能力:数据清洗、统计分析、趋势识别、结果解读
Prompt 预留参数待调用时传入:
- 分析任务描述:${job}
- 待分析的数据:${data}
- 输出类型:${type}
- 输出格式定义:${definition}
Agent URL:https://ai-platform.example.com/api/agent/{agent_id}/{version
```

调用者不需要关心 Agent 内部的 Prompt 怎么写,**只需要知道填什么参数就行。**

Step 2:填充参数调用

在检测代码中,只需填充参数:

```
// 伪代码示意
const result = await aiService.callAI({
  apiKey: 'dataAnalysisApi',  // 从配置中心获取 Agent URL
  sessionId: `task-${taskId}-${timestamp}`,
  variableMap: {
    job: `检查商品的组合计算风险
    背景:商品多规格,活动每件优惠,用户组合购买
    步骤:
    1. 筛选有效规格(库存>0,状态正常)
    2. 找最高价和最低价规格
    3. 获取高价规格基准价(为空则调用工具 - 自定义MCP)
    4. 循环计算组合场景(1个高价+N个低价)
    5. 判断规格到手价是否低于基准价
    6. 记录首次风险件数和最大件数`,
    data: JSON.stringify([
      {
        item_id: "***",
```

```
        skus: [
          {sku_id: "***", page_price: 600, base_price: null, ...},
          {sku_id: "***", page_price: 100, base_price: null, ...}
        ],
        discount: 50
      }
      // ... 更多商品
    ]),
    type: 'JSON',
    definition: `[{
      has_risk: "是否有风险",
      item_id: "商品 id",
      high_sku_id: "风险规格 id",
      low_sku_id: "组合规格 id",
      low_sku_num_min: "最小组合数量",
      loss_amount_min: "最小损失",
      low_sku_num_max: "最大组合数量",
      loss_amount_max: "最大损失",
      ...其他需要的数据
    }]`
  },
  isJsonObject: true
});
```

对比传统方案:

```
传统:150 行代码 + 多层循环 + 复杂逻辑
AI:  自然语言描述任务 + 数据 + 输出定义
```

工作原理:

```
业务参数 → 填入 Agent Prompt 坑位 → LLM 推理计算 → 结构化结果
                    ↑
              AI 平台自动完成
```

关键优势:

- 可读性强:任何人都能看懂业务逻辑;
- 维护简单:规则变化改 `job` 描述即可;
- 快速迭代:修改配置中心即可切换版本。

Step 3:配置中心管理 Agent

将 Agent URL 放在配置中心:

```
{
```

```
  "agentConfig": {
    "dataAnalysisApi": "https://.../agent\_id/version",
    "otherApi": "https://.../agent\_id/version"
  }
}
```

好处:

- 切换 Agent 版本只需改配置,秒级生效;
- 不同环境配置不同版本(灰度发布);
- 问题回滚只需配置回退,无需重新发布代码。

3.3 批量处理策略

实际业务中,一次检测可能有几百甚至几万个数据需要分析。

分批策略:

```
// 将数据分组,每批 3-5 个
const batchSize = 3;
const batches = chunkArray(items, batchSize);
// 批量调用(控制并发)
for (const batch of batches) {
  const result = await aiService.callAI({
    apiKey: 'dataAnalysisApi',
    sessionId: `task-${taskId}-batch${i}`,
    variableMap: {
      job: '...',  // 同样的分析任务
      data: JSON.stringify(batch),  // 这批数据
      type: 'JSON',
      definition: '...'
    }
  });
  // 收集风险数据
  riskItems.push(...result.data);
}
```

为什么分批:

- 准确性:批次太大容易出错或遗漏;
- 稳定性:单次请求失败不影响全局;
- 成本:控制单次 Token 消耗。

实践经验: 数据库初筛百级数据 → 数十批次(每批 3-5 个)→ 总耗时分钟级。

3.4 MCP 工具让 AI 自主获取数据

MCP(Model Context Protocol)的定义和基础概念这里不展开,可查阅相关文档了解。

业务场景:

我们的案例中,商品数据的 `base_price`(基准价)字段经常为空,但这是判断是否超出阈值的关键数据。传统方式需要提前 join 多张表准备好数据,而通过 MCP,可以让 AI 按需获取。

```
// 数据中基准价为空
{
  item_id: "***",
  skus: [{
    sku_id: "***",
    page_price: 600,
    base_price: null,  // ❌ 数据缺失
    ...
  }]
}
```

传统方式:手动注册 MCP

需要手动编写 MCP 工具定义(JSON Schema):

```
{
  "name": "queryBasePriceByItemId",
  "description": "根据商品ID查询基准价",
  "parameters": {
    "type": "object",
    "properties": {
      "item_id": {
        "type": "string",
        "description": "商品ID"
      }
    },
    "required": ["item_id"]
  },
  "implementation": {
    "type": "rpc",
    "service": "com.example.PriceService",
    "method": "queryBasePrice",
    "version": "1.0.0"
  }
}
```

痛点:

- ❌ 手写 Schema 复杂易错
- ❌ RPC 接口参数需要手动映射
- ❌ 接口变更需要同步更新 MCP 定义
- ❌ 缺少统一管理和监控

平台方式:一键生成 MCP ✨

接口平台提供 接口一键转 MCP 能力:

```
步骤 1:平台抓取 RPC 接口
  └ 平台本身支持将内部接口自动转为 HTTP 接口
步骤 2:圈选接口
  └ 在平台找到目标接口
  └ 点击"一键生成 MCP"
步骤 3:自动生成
  └ 自动生成 MCP 工具定义
  └ 自动注册到 Agent 工具箱
  └ 面向各种 AI IDE、Code Agent 开放
```

关键优势:

对比维度	传统手动注册	平台一键生成
Schema 定义	手写 JSON	自动生成
RPC → HTTP	需要自己包装	平台自动转换
参数映射	手动维护	自动解析
接口变更	手动同步	自动更新
监控管理	自己搭建	平台统一管理
接入时间	半天起步	5 分钟

AI 自动调用示例:

在 job 参数中提示 AI 使用工具:

```
variableMap: {
  job: `...
  3. 获取基准价(为空则调用工具 queryBasePriceByItemId)
  ...`
}
```

AI 执行过程:

```
1. 发现 base_price 为 null
2. 发现&调用:获取queryBasePriceByItemId完整定义,以及调用条件(itemId必填)
3. 获取结果:{ base_price: 380 }
4. 继续分析
```

核心价值:

✅ 极简接入:圈选接口,一键生成,5 分钟搞定;

✅ 自动维护:接口变更自动同步,无需手动更新;

✅ 统一管理:平台提供监控、版本管理、权限控制;

✅ 生态开放:面向 AI IDE(Cursor、Windsurf)、Code Agent 等开放。

实践建议:

1. 优先使用平台:如果 RPC 接口已在平台注册,直接一键生成;
2. 工具要稳定:选择稳定的底层服务,避免频繁超时;
3. 设置超时:建议 5-10 秒,避免 AI 分析卡住;
4. 监控调用:在平台查看 MCP 调用情况,及时发现问题。



4.1 开发效率提升

对比维度	传统代码方案	AI 方案
开发周期	3-5 天	1-2 天
代码量	150-200 行	30 行(调用代码)
规则调整	改代码 + 测试 + 发布(1-2 天)	改参数 + 配置切换(1-2 小时)
可读性	需要读代码	自然语言描述

4.2 运行效果

性能数据(百级数据检测):

```
分批策略:数十批次,每批 3-5 个数据
```

```
单批耗时:5-8 秒
总耗时:分钟级
成功率:90%+
准确率:95%+(抽查验证)
```

4.3 业务价值

```
1. 提前发现风险
   ├─ 活动前自动检测
   ├─ 及时发现风险数据
   └─ 避免潜在损失
2. 降低人力成本
   ├─ 原人工排查:数人 × 数天
   ├─ 现自动检测:分钟级
   └─ 人力节省:90%+
3. 提升响应速度
   ├─ 原:活动后被动发现
   ├─ 现:活动前主动预警
   └─ 风险前置:提前数天
```



5.1 参数设计要点

虽然 Agent 的 Prompt 已经预置好,但我们填充的参数(特别是 `job`)质量直接影响效果。

✅ **好的 job 参数:**

```
从用户反馈中提取以下信息:
1. TITLE:简洁的标题(10字以内)
2. PRIORITY:优先级(P0/P1/P2/P3)
3. TAG:问题类型(功能异常/性能问题/体验优化/其他)
要求:准确理解用户意图,标题要能概括核心问题"
```

❌ **常见误区:**

```
1. 太简略:"提取标题" → 标准不明确
2. 太啰嗦:大段描述反而影响理解
```

```
3. 没有约束:输出格式不稳定
```

迭代技巧:

- 先用小样本测试,不断优化 `job` 描述
- 通过调整 `definition` 明确输出格式
- 记录典型 case,形成测试集

5.2 响应处理要鲁棒

AI 平台返回的响应格式可能不一致(不同版本 Agent),需要兼容处理:

```
// 伪代码:兼容多种响应格式
function parseResponse(response) {
  let output = response.data?.content
    || response.data?.runLog?.output
    || response.output;
  // 清理 markdown 包裹
  output = output.replace(/```json\n/g, '').replace(/\n```/g, '');
  // 解析 JSON
  try {
    return JSON.parse(output);
  } catch (e) {
    return {};  // 降级处理
  }
}
```

建议:在 Service 层统一处理,业务代码无需关心细节。

5.3 版本管理策略

AI 平台 Agent 会持续优化,平台会发布新版本。我们通过配置中心管理版本切换。

配置中心灰度:

```
{
  "agentConfig": {
    "dataAnalysisApi": ".../version_stable",      // 稳定版
    "dataAnalysisApi_beta": ".../version_beta"    // 灰度版
  }
}
```

业务代码根据灰度策略选择:

```
// 按比例分流
const apiKey = shouldUseBeta(userId)
  ? 'dataAnalysisApi_beta'
  : 'dataAnalysisApi';
```

快速回滚:

- 发现问题立即修改配置中心
- 将 URL 切回旧版本
- 无需代码发布,秒级生效

5.4 成本控制建议

AI 调用是有成本的(按 Token 计费),需要注意:

1. 数据预处理:调用 AI 前先做初步筛选和清洗
2. 合理分批:批量处理时控制每批大小,避免超限
3. 结果缓存:相同输入缓存结果,避免重复调用
4. 选择合适模型:简单任务用小模型,复杂任务用大模型

实践经验:通过优化后,单次检测成本可控,相比人工成本可忽略不计。

5.5 监控与告警

生产环境必须有监控:

```
关键指标:
├─ 调用成功率
├─ 平均响应时间
├─ 调用量
└─ 成本消耗
告警规则:
├─ 成功率异常 → 立即告警
└─ 失败率超阈值 → 告警
```

建议:接入统一监控平台,自动告警。



6.1 核心收获

技术层面:

- AI 平台大幅降低了 AI 接入门槛(无需从零写 Prompt)
- 预置 Agent + 参数化,快速上手
- 配置中心管理版本,迭代速度快

业务层面:

- 开发效率提升 5 倍(从 5 天到 1 天)
- 维护成本降低 10 倍(2 天 → 2 小时)
- 代码量减少 75%(150 行 → 30 行)

6.2 适用场景

适合用 AI 的场景:

```
✓ 复杂的多步推理和计算(如组合计算分析)
✓ 规则频繁变化(业务规则调整)
✓ 代码实现复杂但业务逻辑清晰
✓ 数据分析和异常检测
✓ 需要动态获取数据(通过 MCP 工具)
✓ 数据源分散,难以提前 join
```

不适合的场景:

```
✗ 实时性要求极高(毫秒级响应)
✗ 简单的阈值判断(if-else 就能搞定)
✗ 高频调用且成本敏感
✗ 对准确性要求 100% 的场景(金融核算等)
```

判断标准:

```
传统代码行数 > 100 行? → 考虑 AI
规则每月变化 > 2 次? → 考虑 AI
需要多步推理计算? → 考虑 AI
对成本不敏感? → 考虑 AI
```

6.3 一点思考

关于 AI 的定位:

AI 不是万能的,但在合适的场景下确实能带来质的提升。关键是找到 AI 的优势场景,与传统技术互补,而不是全面替代。

关于 AI 平台的价值:

AI 平台采用"预置 Agent + 参数化"的模式,最大的价值是降低了 AI 应用的门槛:

- 不需要深入了解 AI 技术
- 不需要从零写 Prompt
- 不需要调试模型参数

原本只有 AI 专家能做的事,现在业务开发也能快速上手。这种"能力平台化"的思路,才是 AI 真正赋能业务的方式。

选对了场景,用对了工具,AI 就能快速产生业务价值。



本文作者影子，来自淘天集团–天猫技术团队。天猫技术是阿里巴巴旗下专注于电商场景的综合技术团队，不仅服务于自身电商业务增长，也在不断探索如何用AI、大数据、交互技术赋能行业交易、品牌营销与消费者体验，成为全球领先的品质购物平台和技术引擎。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 8235

留言

写留言

还没有留言，来写首评